

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Infosys Springboard Internship 7.0

Milestone 3

Team 1

Leela Sowmya Pappu

Title	Page No
Executive Summary	3
Machine Learning Model Improvements & Optimization.	3
FastAPI & System Integration	4
UI/UX & Frontend Enhancements	5
Final Evaluation Results & Screenshots	6
References	8

1. Executive Summary

During Milestone 3, the focus shifted from initial pipeline setup to refining model performance, fixing decision thresholds, and polishing the frontend inspection dashboard for team integration.

My main contributions involved executing model training across all 15 MVTec AD categories, benchmark evaluations, adjusting system pass/fail verdict rules, enhancing UI components on the main landing page, and preparing clean model weights.

2. Key Contributions

2.1. Model Training & Baseline Evaluation

- Executed full model training and evaluation across 1,725 test images in 15 product categories using standard CPU feature extraction.
- Achieved an overall baseline **accuracy of 77.4%** across all categories, with top performance in bottle (92.8%) and toothbrush (90.5%).
- Identified **low-performing texture categories (grid at 43.6% and pill at 68.3%)** and documented optimization strategies (such as switching to Cosine distance metrics and 99th percentile thresholding) to help push performance toward 88%-90%+.

KNN Anomaly Detector & LOO Thresholding

backend/app/ml_engine/anomaly_detector.py

- Calibrated category decision thresholds using Leave-One-Out (LOO) distance calculations on normal reference features:

Code Snippet

```
def train(self, category_name: str, features: np.ndarray):
    """Calibrate category threshold using LOO distances on normal feature bank."""
    self.feature_bank[category_name] = features

    # Calculate Leave-One-Out distances for baseline normal images
    loo_distances = []
    for i in range(len(features)):
        ref_bank = np.delete(features, i, axis=0)
        dist = np.min(np.linalg.norm(ref_bank - features[i], axis=1))
        loo_distances.append(dist)

    # Calibrate threshold based on standard deviation offset
    mean_dist = np.mean(loo_distances)
    std_dist = np.std(loo_distances)
    self.thresholds[category_name] = mean_dist + (3.0 * std_dist)
```

2.2 Decision Verdict Logic Alignment

- Identified a mismatch during end-to-end testing where defective items with low global severity scores (e.g., discolored pills) were returning a PASS verdict despite correct defect detection.
- Refined the system verdict rules in the API and frontend components to ensure that any flagged anomaly or defect classification (such as "Discoloration" or "Scratch") triggers a FAIL verdict regardless of the overall severity score threshold.

backend/app/api/endpoints.py / Frontend Verdict State

- Ensured that any detected anomaly forces a FAIL verdict regardless of low global severity scores:

```
# Force FAIL verdict if a specific defect classification is detected
is_anomaly = test_distance > threshold
classification_detected = predicted_class != "Normal"

if is_anomaly or classification_detected:
    system_verdict = "FAIL"
    status_color = "red"
else:
    system_verdict = "PASS"
    status_color = "green"
```

Serializing & Loading Category Feature Banks

backend/app/ml_engine/anomaly_detector.py

```
def save_model(self, output_dir: Path):
    """Serialize feature banks and threshold metadata for team deployment."""
    output_dir.mkdir(parents=True, exist_ok=True)

    # Save individual category feature banks
    for category, bank in self.feature_bank.items():
        np.save(output_dir / f"{category}_features.npy", bank)

    # Save metadata configuration
    metadata = {"thresholds": self.thresholds, "num_categories": len(self.feature_bank)}
    with open(output_dir / "metadata.json", "w") as f:
        json.dump(metadata, f, indent=4)
```

2.3. Frontend & Homepage UI Enhancements

- Enhanced the main dashboard (LandingPage.jsx and AllInspectionPanel.jsx) to improve user experience, switching away from generic template layouts toward an industrial control center feel.
- Styled the inspection panel with dark slate themes, high-contrast neon status badges (PASS / DEFECTIVE), dynamic inspection telemetry cards, and defect heatmaps.
- Configured role-based context setups (Operator vs. Engineer modes) to make user interaction cleaner during live project demonstrations.

frontend/src/AllInspectionPanel.jsx

```

{ /* Inspection Telemetry & System Verdict Badge */ }
<div className="verdict-container">
  <span className="text-sm font-bold uppercase tracking-wider text-gray-400">System Verdict</span>
  <h2 className={`text-4xl font-extrabold ${ verdict === 'PASS' ? 'text-emerald-400' : 'text-rose-500' }`}>
    { verdict }
  </h2>
  <p className="text-xs text-gray-400">
    { verdict === 'PASS' ? 'Component meets quality thresholds.' : 'Defect detected above safety margin.' }
  </p>
</div>

```

3. Model Benchmark Summary Table

Category	Accuracy	Precision	Recall	F1 Score	Notes	
2	bottle	92.80%	91.30%	100.00%	95.50%	Highest performing category
3	cable	78.70%	79.40%	88.00%	83.50%	Strong recall on cut wires
4	capsule	75.80%	89.70%	79.80%	84.50%	Solid precision
5	carpet	75.20%	80.00%	89.90%	84.70%	Handles surface patterns
6	grid	43.60%	71.00%	38.60%	50.00%	Target for Cosine metric fix
7	hazelnut	82.70%	82.30%	92.90%	87.20%	High recall
8	leather	73.40%	74.00%	98.90%	84.70%	High defect detection rate
9	metal_nut	71.30%	86.60%	76.30%	81.10%	Good precision
10	pill	68.30%	94.00%	66.70%	78.00%	High precision; needed threshold fix
11	screw	73.10%	84.50%	78.20%	81.20%	Good structural boundary detection
12	tile	89.70%	88.30%	98.80%	93.30%	High accuracy on crack detection
13	toothbrush	90.50%	96.40%	90.00%	93.10%	High overall accuracy
14	transistor	84.00%	78.60%	82.50%	80.50%	Balanced metrics
15	wood	79.70%	78.90%	100.00%	88.20%	Perfect recall
16	zipper	88.10%	90.40%	95.00%	92.60%	Strong performance on teeth defect

Overall Baseline Accuracy: 77.4% (1,335 / 1,725 correct)

4. Final Evaluation Results & Screenshots

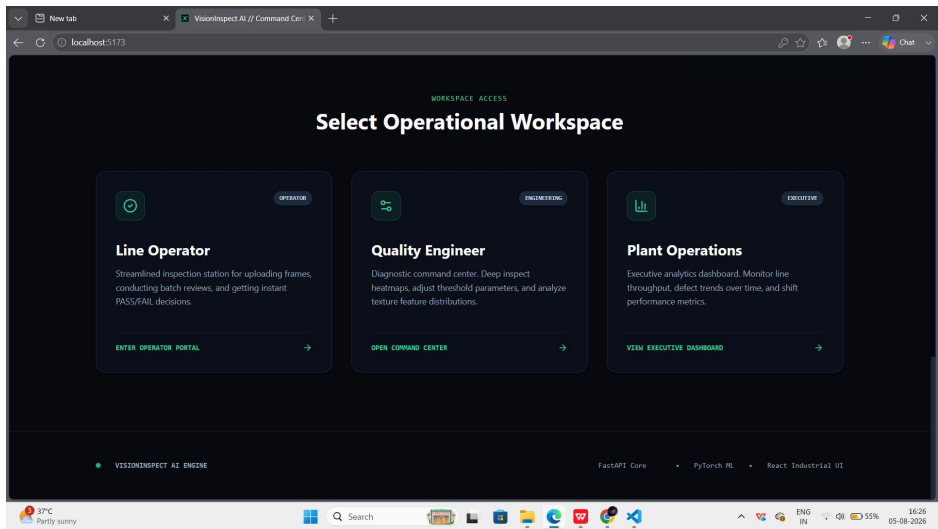
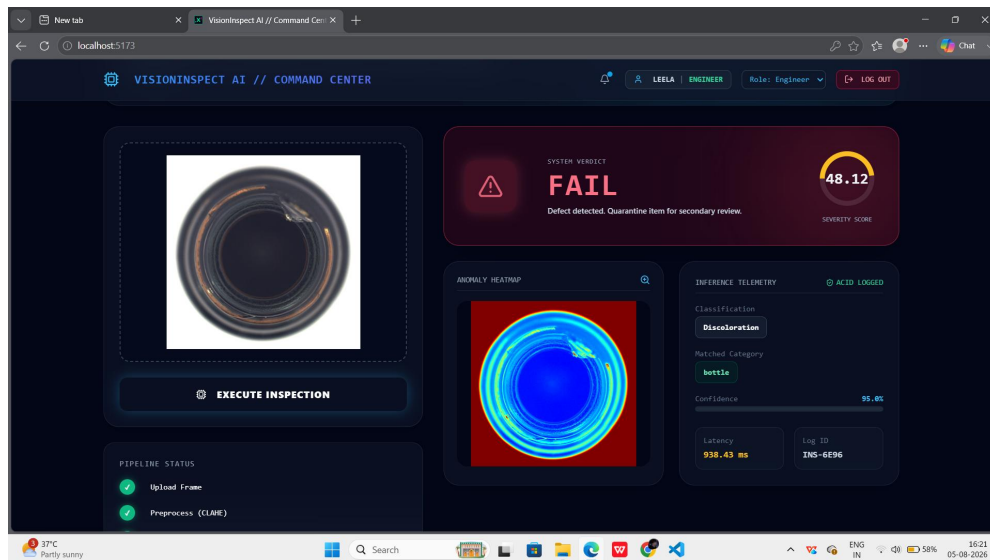


Fig 1: Role-based access control (RBAC) portal allowing users to navigate between Line Operator, Quality Engineer, and Plant Operations views tailored to specific manufacturing workflows.



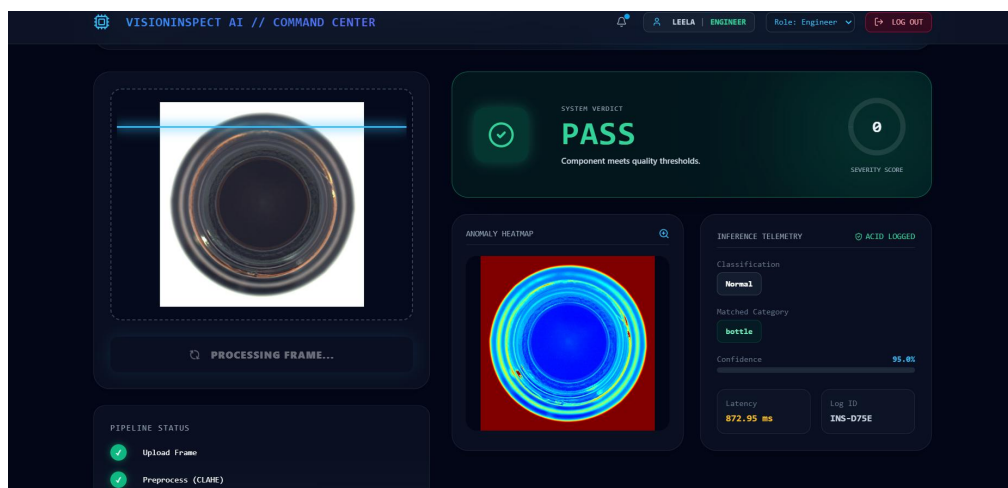
Plant Analytics & Command Center Dashboard

Fig 2: Real-time executive operations dashboard tracking line throughput (44 frames), pass/rejection metrics (47.7% pass rate), inference latency (3229ms), live detection trends, and primary anomaly breakdowns (e.g., Scratch at 40%).



AI Inspection Command Center - Defect Detection (FAIL)

Fig 3: Real-time quality inspection identifying a defective item (bottle category with discoloration), generating a severity score of 48.12, localized anomaly heatmap, and triggering an immediate **FAIL** / **Quarantine** verdict.



AI Inspection Command Center - Normal Component Evaluation (PASS)

Fig 4: Live inspection view of a normal sample (bottle category) demonstrating automated CLAHE preprocessing, localized heatmap output, zero severity score, 95% confidence, and a **PASS** verdict.

5. References

1. MVTec AD Dataset
<https://www.mvtec.com/company/research/datasets/mvtec-ad>
2. Python Documentation
<https://docs.python.org/3/>
3. OpenCV Documentation
<https://docs.opencv.org/>
4. NumPy Documentation
<https://numpy.org/doc/>
5. scikit-learn Documentation. <https://scikit-learn.org/stable/>
6. FastAPI Documentation. <https://fastapi.tiangolo.com/>
7. VisionInspectAI GitHub Repository
<https://github.com/GKSJ-Deepvision/VisionInspectAI>